

Signature transformers for calibrating rough volatility models

Abstract

In this paper, we present a new model, named the SigFormer model, based on both the self-attention mechanism and the signature transform to calibrate volatility processes simulated by various rough volatility models. We also provide numerical examples to demonstrate its advantages in the calibration of both single and multiple parameters. Code is available at github.com/changanluoxue/signature-transform.

1 Introduction

The effective calibration for rough volatility models represents a crucial challenge in the field of mathematical finance. Over the years, various neural networks based calibration approaches have been developed. These approaches can be categorized by the different types of neural networks they are based. Since the fully connected feed forward neural network is the basic neural network model, there are many references use it directly for calibration (see e.g., Horvath, Muguruza and Tomas (2021), Rosenbaum and Zhang (2021)). The fully connected feed forward neural network can also be integrated into the stochastic Volterra integral equations to calibrate more precisely (see e.g., Wang and Hong (2021), Ma, J., Wu, X., and Wu, H. (2023)). Furthermore, Stone (2020) verified that the convolutional neural network can also be used for calibrating rough volatility models. Kidger, Bonnier, Perez Arribas, Salvi and Lyons (2019) introduced the signature based neural network to calibrate fractional Brownian motions.

Recently, large-scale natural language processing models (LLMs) have been developed rapidly thanks to the transformer model proposed by Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser L. and Polosukhin, I. (2017). It is nature to think that whether the transformer can improve the performance of existing neural networks based calibration approaches since the volatility data is also sequential. The aim of this paper is to explore a appropriate combination method for transformer and existing neural networks based calibration approaches to calibrate rough volatility models.

In this paper, we use the self-attention mechanism with signature transform as a layer in a neural network to investigate whether this architecture can help to calibrate parameters from a set of rough paths and we follow Tong (2023) to call

it SigFormer model. The structure of the paper is the following. In Section 2, we give a brief introduction to the rough volatility models, path signature theory and attention mechanism that we will use in this paper. The specific architecture of our SigFormer model is clarified in Section 3. In Section 4, we begin with a simple SigFormer model, only consisting of the signature transform layer and single-head attention layer, to present the evolution leading to our SigFormer model. Finally, we discuss our conclusions in Section 5.

2 Background

This section offers a brief but necessary introduction to the rough volatility models, path signature theory and attention mechanism.

2.1 Rough volatility processes

We begin this section by introducing the fractional Brownian motion (fBm), before moving on to describe the rough Bergomi (rBergomi) volatility process and rough Heston (rHeston) volatility process.

Fractional Brownian motion. There are many different integral representations of fBm, here we introduce the characterisation provided by Mandelbrot and Van Ness (1968) as

$$\begin{aligned} W_t^H &= \frac{1}{\Gamma(\frac{1}{2} + H)} \int_{-\infty}^0 \left((t-s)^{H-\frac{1}{2}} - (-s)^{H-\frac{1}{2}} \right) dB_s \\ &+ \frac{1}{\Gamma(\frac{1}{2} + H)} \int_0^t (t-s)^{H-\frac{1}{2}} dB_s, \quad t \in [0, T] \end{aligned} \quad (1)$$

where T is a positive constant, $H \in (0, 1)$ represents the Hurst parameter, B is a standard Brownian motion and Γ is the standard Gamma function. It is hard to simulate the paths of fBm, as (1) contains an integral process and the kernel $(t-s)^{H-\frac{1}{2}}$ with $H \in (0, \frac{1}{2})$ is singular at point $s = t$.

Rough Bergomi volatility process. The rBergomi volatility process is defined as

$$V_t^{rBergomi} = V_0^{rBergomi} \exp \left(\eta \sqrt{2H} Z_t - \frac{\eta^2}{2} t^{2H} \right), \quad t \in [0, T], \quad (2)$$

with

$$Z_t = \int_0^t (t-s)^{H-\frac{1}{2}} dB_s, \quad (3)$$

where $V_0^{rBergomi}$ and η are both strictly positive constant.

Rough Heston volatility process. The rough Heston volatility process is defined as

$$V_t^{rHeston} = V_0^{rHeston} + \frac{1}{\Gamma(\frac{1}{2} + H)} \int_0^t (t-s)^{H-\frac{1}{2}} \kappa_1 (\theta - V_s^{rHeston}) ds$$

$$+ \frac{1}{\Gamma(\frac{1}{2} + H)} \int_0^t (t-s)^{H-\frac{1}{2}} \kappa_2 \sqrt{V_s^{rHeston}} dB_s, \quad t \in [0, T], \quad (4)$$

where $\kappa_1, \kappa_2, \theta$ are all constants.

We note that there is only one parameter H in fBm while there are two and four parameters in rBergomi volatility process (i.e. H, η) and rHeston volatility process (i.e. $H, \kappa_1, \kappa_2, \theta$), respectively.

2.2 Signature

Given a continuous path $X = \{x_t | x_t = (x_t^1, \dots, x_t^d) \in \mathbb{R}^d, t \in [a, b]\}$ with $a, b \in \mathbb{R}$ and $d \in \mathbb{N}^+$, we can define its signature as the collection of iterated integrals

$$\begin{aligned} \text{Sig}(X) &= \left(\int_{a < t_1 < \dots < t_i < b} dx_{t_1} \otimes \dots \otimes dx_{t_i} \right)_{i \geq 1} \\ &= \left(\left(\int_{a < t_1 < \dots < t_i < b} dx_{t_1}^{d_1} \dots dx_{t_i}^{d_i} \right)_{1 \leq d_1, \dots, d_i \leq d} \right)_{i \geq 1}, \end{aligned} \quad (5)$$

where $\otimes$ denotes the tensor product. From Equation (5), we can easily define the i -th signature of path X as

$$\text{Sig}_i(X) = \int_{a < t_1 < \dots < t_i < b} dx_{t_1} \otimes \dots \otimes dx_{t_i}, \quad i = 1, 2, \dots \quad (6)$$

where the "zeroth" term $\text{Sig}_0(X)$, by convention, is equal to 1. With these notations, we can give the definition as follows.

Definition 2.1 (Signature). *The signature of a continuous path $X : [a, b] \mapsto \mathbb{R}^d$ is defined as an infinite series of tensors indexed by the signature order $i \in \mathbb{N}$,*

$$\text{Sig}(X) := (1, \text{Sig}_1(X), \text{Sig}_2(X), \dots, \text{Sig}_i(X), \dots),$$

where $\text{Sig}_i(X)$ for $i = 1, 2, \dots$ are defined as Equation (6).

Furthermore, we can also define the corresponding truncated signature as

$$\begin{aligned} \text{Sig}^N(X) &= \left(\int_{a < t_1 < \dots < t_i < b} dx_{t_1} \otimes \dots \otimes dx_{t_i} \right)_{1 \leq i \leq N} \\ &= (\text{Sig}_1^N(X), \dots, \text{Sig}_N^N(X)), \end{aligned} \quad (7)$$

with the truncation order $N \in \mathbb{N}^+$ and we exclude the constant "zeroth" term here.

Remark 2.1. We interpret a stream of data $X^n = (x_{t_1}, \dots, x_{t_n})^T \in \mathbb{R}^{n \times d}$ as a discretisation of path X on $n \in \mathbb{N}^+$ time grids $a = t_1 < \dots < t_n = b$. Then the (truncated) signature of X^n is defined as the (truncated) signature of its linear interpolation.

Remark 2.2. The signature $\text{Sig}^N(X^n)$ has

$$\sum_{i=1}^N d^i = \frac{d^{N+1} - d}{d - 1}$$

components. Obviously, the number of components does not depend on the length of data. Thus the signature is an excellent way to tackle long streams of data.

There are some good properties of signature that make it attractive when dealing with streams of data and here we present these without proof.

Proposition 2.1 (Uniqueness (Hambly and Lyons (2010))). *Given the time-augmented path as $\hat{X}^n := ((x_{t_1}, t_1), \dots, (x_{t_n}, t_n))^T \in \mathbb{R}^{n \times (d+1)}$. Then $\text{Sig}(\hat{X}^n)$ can uniquely determines $\hat{X}^n$.*

The Proposition 2.1 means that there is no information lost during the signature transform. The reason why we need the time-augmented path instead of just the path is the signature transform is invariance to time reparameterisations, which means the signature transform only encodes the order in which data arrives without caring precisely when it arrives.

Proposition 2.2 (Factorial decay (Lyons (1998))). *There exists a constant $C(X^n)$ depending on path X^n such that*

$$\|\text{Sig}_i(X^n)\| \leq \frac{C(X^n)^i}{i!},$$

where $\|\cdot\|$ is any tensor norm on $(\mathbb{R}^d)^{\otimes i}$.

The Proposition 2.2 indicate that the terms of the signature decay in size factorially. This ensures that the most significant contributions are given by the lower order terms which makes it reasonable to use truncated signature.

2.3 Transformer and attention mechanism

Transformer is first proposed by Vaswani, A., Shazeer, N., Parmar, N., Uszkor-eit, J., Jones, L., Gomez, A. N., Kaiser L. and Polosukhin, I. (2017) for natural language processing. The key component in transformer is self-attention mechanism.

Self-attention mechanism. Consider the discretized path $X^n \in \mathbb{R}^{n \times d}$, and let $W_Q \in \mathbb{R}^{d_{\text{attn}} \times d}$, $W_K \in \mathbb{R}^{d_{\text{attn}} \times d}$ and $W_V \in \mathbb{R}^{d_{\text{attn}} \times d}$ with $d_{\text{attn}} \in \mathbb{N}^+$. Then we can define the query, key and value in self-attention mechanism as $Q =$

$X^n W_Q^T$, $K = X^n W_K^T$ and $V = X^n W_V^T$, respectively, which are linear projections from the path. Then the self-attention mechanism function can be defined as

$$\text{Self-Attention}(X^n; W_Q, W_K, W_V) := \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V. \quad (8)$$

Intuitively, the self-attention mechanism function can be viewed as the combination of linear and non-linear transformation applied to the input X^n which is similar to the multilayer perceptron (MLP).

Instead of performing a single self-attention mechanism function, it is beneficial to apply the multi-head self-attention mechanism function which performs the self-attention mechanism function in parallel and then linearly projects these concatenated outputs into an appropriate space.

Multi-head attention mechanism. The multi-head self-attention mechanism function can be defined as

$$\begin{aligned} \text{Multi-Head}(X^n; W_{Q_1}, \dots, W_{Q_h}, W_{K_1}, \dots, W_{K_h}, W_{V_1}, \dots, W_{V_h}, W_O) \\ := \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O, \end{aligned} \quad (9)$$

where W_O denotes the weight of the output projection, $h \in \mathbb{N}^+$ denotes the number of heads and head_i for $i = 1, \dots, h$ are defined by the Equation (8) with $X_i^n \subseteq X^n$, $W_{Q_i}, W_{K_i}, W_{V_i} \in \mathbb{R}^{d_{\text{attn}_i} \times d}$ with $d_{\text{attn}_i} \in \mathbb{N}^+$.

Multi-head self-attention mechanism allows the model to jointly attend to information from different representation subspaces at different positions while averaging inhibits this with single-head self-attention mechanism.

3 Methodology

In this section, we will clarify the specific architecture of our signature transformer (SigFormer) model, and the reasons why we chose this structure will be explained in the numerical experimental section.

3.1 Augmented with convolutional layer

Let's consider a discretized path $X^n = (x_{t_1}, \dots, x_{t_n})^T \in \mathbb{R}^{n \times d}$. According to Kidger, Bonnier, Perez Arribas, Salvi and Lyons (2019), it is more effective to do the feature map before taking the signature. Then, on the top of our SigFormer model, a convolutional layer $\Phi(X^n; W^\Phi, B^\Phi) : \mathbb{R}^{n \times d} \mapsto \mathbb{R}^{\tilde{n} \times \tilde{d}}$ with kernel size $k \in \mathbb{N}^+$ and stride $s \in \mathbb{N}^+$ is applied to perform the feature map

$$\Phi(X^n; W^\Phi, B^\Phi) = W^\Phi \star X^n + \mathbf{1}_{\tilde{n}} B^\Phi, \quad (10)$$

with

$$(W^\Phi \star X^n)_{i,j} = \sum_{i_1=1}^k \sum_{i_2=1}^d W_{j,i_1,i_2}^\Phi X_{s(i-1)+i_1,i_2}^n, \quad i = 1, \dots, \tilde{n}, j = 1, \dots, \tilde{d}, \quad (11)$$

where $\tilde{n} \in \mathbb{N}^+$ and $\tilde{d} \in \mathbb{N}^+$ represent the output dimension, $\mathbf{1}_{\tilde{n}} \in \mathbb{R}^{\tilde{n}}$ is all-ones column vector, $\star$ is the valid cross-correlation operator and $W^\Phi \in \mathbb{R}^{\tilde{d} \times k \times d}$ and $B^\Phi \in \mathbb{R}^{\tilde{d}}$ are weight and bias for the convolutional layer, respectively. Here, we can always find an appropriate $\tilde{n}$ by doing the zero padding.

After this convolutional layer, according to Proposition 2.1, it is beneficial to augment the output with time before taking the signature. And it can also be augmented with the original data X^n when we set $\tilde{n} = n$.

3.2 Taking the signature with lifted data

Taking signature transform will consume the stream-like nature of the data. To explain further, consider a stream of data $Y^{\tilde{n}} = (y_{t_1}, \dots, y_{t_{\tilde{n}}})^T \in \mathbb{R}^{\tilde{n} \times \tilde{d}}$ to be the output of the convolutional layer and already been augmented, here $\tilde{d}$ denotes the number of features and $\tilde{n}$ denotes the length of sequence which indicates there is a relative position for each element in this data. If we take the truncated signature for $Y^{\tilde{n}}$ with truncation order N , then, according to Remark 2.2, the length of output $\text{Sig}^N(Y^{\tilde{n}}) \in \mathbb{R}^{\frac{\tilde{d}^{N+1}-\tilde{d}}{\tilde{d}-1}}$ is not related to $\tilde{n}$ i.e. it is not a sequential data. The solution is to lift the data before taking the signature as follows

$$l(Y^{\tilde{n}}) = (Y_{1:2}^{\tilde{n}}, \dots, Y_{1:i}^{\tilde{n}}, \dots, Y_{1:\tilde{n}}^{\tilde{n}}), \quad (12)$$

where $Y_{1:i}^{\tilde{n}} = (y_{t_1}, \dots, y_{t_i})^T \in \mathbb{R}^{i \times \tilde{d}}$ for $i = 1, \dots, \tilde{n}$. Each element in the lifted data $l(Y^{\tilde{n}})$ is sequential data. Then the output of taking the signature with lifted data is

$$\text{Sig}^N(l(Y^{\tilde{n}})) = (\text{Sig}^N(Y_{1:2}^{\tilde{n}}), \dots, \text{Sig}^N(Y_{1:\tilde{n}}^{\tilde{n}}))^T. \quad (13)$$

Now the output $\text{Sig}^N(l(Y^{\tilde{n}})) \in \mathbb{R}^{(\tilde{n}-1) \times \frac{\tilde{d}^{N+1}-\tilde{d}}{\tilde{d}-1}}$ has another dimension related to $\tilde{n}$ i.e. it is a sequential data with stream-like nature. Furthermore, we chose the stride equals to 1 in Equation (12) but it is also naturally to choose a larger stride to decrease the length of $l(Y^{\tilde{n}})$. There are also some other plausible choices for l such as taking a sliding window.

3.3 Self-attention layer

The signature can be consider as a feature map which helps to extract the information from the input data. So there is still something needed to process the output after taking the signature, and the feed forward neural network (FNN) is a alternate choice as Kidger, Bonnier, Perez Arribas, Salvi and Lyons (2019). Besides FNN, we choose to add another self-attention layer because it will help in multiple parameters calibration and we will verify this in the next numerical experiments.

Consider the output of taking the signature as follows

$$\text{Sig}^N(l(Y^{\tilde{n}})) = (\text{Sig}^N(Y_{1:2}^{\tilde{n}}), \dots, \text{Sig}^N(Y_{1:\tilde{n}}^{\tilde{n}}))^T$$

$$= \begin{pmatrix} \text{Sig}_1^N(Y_{1:2}^{\tilde{n}}) & \cdots & \text{Sig}_N^N(Y_{1:2}^{\tilde{n}}) \\ \vdots & \ddots & \vdots \\ \text{Sig}_1^N(Y_{1:\tilde{n}}^{\tilde{n}}) & \cdots & \text{Sig}_N^N(Y_{1:\tilde{n}}^{\tilde{n}}) \end{pmatrix}_{(\tilde{n}-1) \times \frac{\tilde{d}^{N+1}-\tilde{d}}{\tilde{d}-1}}. \quad (14)$$

We employ a self-attention mechanism function as Equation (8), then the output become $\text{Self-Attention}(\text{Sig}^N(l(Y^{\tilde{n}})))$, here we omit the weight matrix for simplicity. As we discussed in Section 2.3, it is beneficial to employ a multi-head attention mechanism function instead of a single-head one. Thus, we use a N -head attention mechanism function and each head corresponds to a different signature level as follows

$$\text{Multi-Head}(\text{Sig}^N(l(Y^{\tilde{n}}))) = \text{Concat}(\text{head}_1, \dots, \text{head}_N) \widetilde{W}_O, \quad (15)$$

where $\text{head}_i = \text{Self-Attention}((\text{Sig}_i^N(Y_{1:2}^{\tilde{n}}), \dots, \text{Sig}_i^N(Y_{1:\tilde{n}}^{\tilde{n}}))^T)$ for $i = 1, \dots, N$. Here we also omit all the weight matrix except the linear projection matrix $\widetilde{W}_O$ for output. After the self-attention layer, we need another feedforward neural network f on the bottom to project the output to a appropriate dimension.

Now we can clarify the specific architecture of our SigFormer model as below and we also present it in Figure 1.

- a convolutional layer with 3 channels and kernel size 3;
- augmented with time and original values;
- lifted as Equation (12) with stride equals to 10;
- taking the signature transform with truncation order $N = 3$;
- the multi-head self-attention layer with the number fo heads $h = N = 3$ and each different head corresponds to a different signature level;
- a fully connected feedforward neural network with 5 hidden layers, each of size 32 and ReLU activation;
- a non-linear transform with sigmoid function.

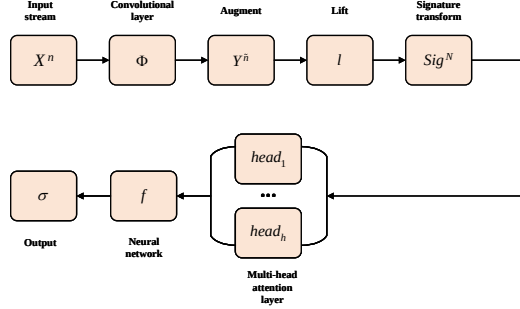


Figure 1: Structure of the SigFormer model.

4 Numerical experiments

Now we will move on to perform some numerical experiments to verify the advantages of this architecture in Section 3 and also to explain why we chose it. Here we choose another two neural network based models for comparison. The first one is a convolutional neural network (CNN) model proposed by Stone (2020) and the corresponding source code is publicly available on Github¹. The other one is a signature-based (DeepSigNet) model proposed by Kidger, Bonnier, Perez Arribas, Salvi and Lyons (2019) with the code on Github². We also present the specific architecture of these two models we used in numerical experiments on Appendix A.1. In the following experiments, we will show results of CNN and DeepSigNet model as benchmarks. And we will show the number of model parameters instead of the program runtime to represent the model complexity, in order to avoid the impact of different computational hardware. This section is split into two subsections to discuss the single parameter calibration problem and the multiple parameters calibration problem, respectively.

4.1 Calibrating the Hurst parameter with rough volatility processes

In this section, we perform two examples to verify the advantages of our SigFormer model for single parameter calibration problems and also determine the optimal signature truncation order.

¹<https://github.com/henrymstone/rough-calibration-using-CNNS>

²<https://github.com/patrick-kidger/Deep-Signature-Transforms>

4.1.1 Implementation details

In order to generate data sets containing paths with different roughness, we first sample five Hurst parameters H from different probability distributions: the Uniform distribution³ on $(0, 0.5)$ and the Beta(1, 9) distribution⁴. Obviously, paths with H in the Beta distributed set is rougher than those with H in uniformly distributed set. Then we use these different H to simulate fBm, rBergomi and rHeston paths with different number of time steps $\{100, 200, 300, 400, 500\}$ to generate ten corresponding training/test data sets with the same size as shown in Table 1.

Table 1: Training and test data size.

Data set	Number of samples
Training set	3000
Test set	1000

We employ the Cholesky decomposition to simulate fBm and then obtain rBergomi paths with $\eta = 1$ and $V_0^{rBergomi} = 0.01$; this very well-known simulation technique is recommended because the resulting sample paths have the exact distribution, rather than an approximate distribution, of the normalised log volatility process of the rBergomi volatility process. And we apply the fast algorithm proposed by Ma and Wu (2022) with $\kappa_1 = 0.1$, $\kappa_2 = 0.03$, $\theta = 0.3$, $V_0^{rHeston} = 0.01$ to simulate rHeston paths.

We use adaptive moment estimation (Adam) optimizer to train these models, setting *batch size* = 64, *epochs* = 100 and *learning rate* = 0.0001 as is fairly standard. We use the mean square error (MSE) as the loss function for training and provide the root mean square error (RMSE) in results as reference.

4.1.2 Calibrating the Hurst parameter from volatility paths with different lengths

In order to validate the necessity of both the convolutional and the fully connected layers in our SigFormer model as well as show our SigFormer is beneficial from multi-head attention, we propose another simple signature-based transformer for comparison, the architecture of this simple SigFormer model is also presented in Appendix A.1. Estimating the Hurst parameter H of a fBm, rBergomi or rHeston path is considered a nontrivial task because of the paths' non-stationarity and long range dependencies. Therefore paths with more time steps and smaller H (i.e. rougher) are more difficult to calibrate.

In this example, we train a variety of models that we mentioned before to perform this estimation. That is, to learn the map $(t^n, X^n) \mapsto H$, where t^n de-

³The corresponding set of possible H values is $\{0.06, 0.12, 0.27, 0.35, 0.45\}$.

⁴The corresponding set of possible H values is $\{0.04, 0.08, 0.11, 0.16, 0.21\}$.

notes the time steps and X^n are the corresponding simulated paths. The results are shown in Table 2 for $H \sim \text{Uniform}(0, 0.5)$ and Table 3 for $H \sim \text{Beta}(1, 9)$ with the corresponding loss plots shown in Appendix B.1. In order to present the results more clearly, we also plot Table 2 and Table 3 into Figure 2. From these results, we can easily conclude that the SigFormer model tends to outperform the competing models in three scenarios for single parameter calibration problems: (a) paths with more time steps; (b) rougher paths; (c) paths simulated by more complex volatility models. We attribute this to the self-attention mechanism, which enhances the capacity of the SigFormer model for analyzing information from path signatures, thereby making it outperforming in complex calibration problems. We also observe that the number of parameters i.e. the complexity for the SigFormer model increases with the input length while the Simple SigFormer and DeepSigNet model maintain a constant number of parameters. This feature of the SigFormer model is not directly from introducing the multi-head self-attention layer, but rather through the lifting operation before taking the signature. Furthermore, we need to take the signature multiple times since the lifting operation preserves the stream-like nature of the data, which will increase the input length of the multi-head self-attention layer and subsequently increase the number of model parameters.

Table 2: Test results for different models with input lengths varying from 100 to 500 and $H \sim \text{Uniform}(0, 0.5)$.

Models	Input length	Test RMSE			#Params
		fBm	rBergomi	rHeston	
DeepSigNet	100	1.65×10^{-2}	7.20×10^{-2}	1.39×10^{-1}	9261
	200	1.07×10^{-2}	5.77×10^{-2}	1.21×10^{-1}	9261
	300	1.57×10^{-2}	3.91×10^{-2}	1.40×10^{-1}	9261
	400	1.14×10^{-2}	5.22×10^{-2}	1.19×10^{-1}	9261
	500	2.63×10^{-2}	4.80×10^{-2}	1.34×10^{-1}	9261
CNN	100	7.53×10^{-2}	4.89×10^{-2}	1.00×10^{-1}	255073
	200	7.80×10^{-2}	5.24×10^{-2}	8.34×10^{-2}	320609
	300	9.23×10^{-2}	4.43×10^{-2}	1.01×10^{-1}	386145
	400	7.92×10^{-2}	3.78×10^{-2}	1.00×10^{-1}	435297
	500	9.05×10^{-2}	5.12×10^{-2}	8.70×10^{-2}	500833
SigFormer	100	1.34×10^{-2}	5.30×10^{-2}	9.63×10^{-2}	126906
	200	1.38×10^{-2}	4.77×10^{-2}	7.34×10^{-2}	176506
	300	1.28×10^{-2}	4.12×10^{-2}	4.45×10^{-2}	226106
	400	1.42×10^{-2}	4.00×10^{-2}	4.29×10^{-2}	275706
	500	1.27×10^{-2}	3.69×10^{-2}	3.77×10^{-2}	325306
Simple SigFormer	100	6.83×10^{-2}	1.47×10^{-1}	1.44×10^{-1}	6666
	200	7.03×10^{-2}	1.42×10^{-1}	1.43×10^{-1}	6666
	300	6.93×10^{-2}	1.44×10^{-1}	1.46×10^{-1}	6666
	400	6.69×10^{-2}	1.42×10^{-1}	1.41×10^{-1}	6666
	500	6.32×10^{-2}	1.42×10^{-1}	1.45×10^{-1}	6666

Table 3: Test results for different models with input lengths varying from 100 to 500 and $H \sim \text{Beta}(1, 9)$.

Models	Input length	Test RMSE			#Params
		fBm	rBergomi	rHeston	
DeepSigNet	100	1.31×10^{-2}	5.55×10^{-2}	6.04×10^{-2}	9261
	200	1.14×10^{-2}	4.68×10^{-2}	5.83×10^{-2}	9261
	300	1.58×10^{-2}	4.77×10^{-2}	5.82×10^{-2}	9261
	400	2.44×10^{-2}	4.23×10^{-2}	5.84×10^{-2}	9261
	500	2.24×10^{-2}	3.83×10^{-2}	5.83×10^{-2}	9261
CNN	100	1.39×10^{-1}	5.68×10^{-2}	8.57×10^{-2}	255073
	200	9.24×10^{-2}	5.24×10^{-2}	8.44×10^{-2}	320609
	300	9.39×10^{-2}	6.30×10^{-2}	8.78×10^{-2}	386145
	400	8.87×10^{-2}	5.23×10^{-2}	7.86×10^{-2}	435297
	500	9.18×10^{-2}	5.32×10^{-2}	7.60×10^{-2}	500833
SigFormer	100	1.38×10^{-2}	3.76×10^{-2}	5.83×10^{-2}	126906
	200	1.81×10^{-2}	3.68×10^{-2}	5.17×10^{-2}	176506
	300	1.81×10^{-2}	3.63×10^{-2}	4.77×10^{-2}	226106
	400	1.85×10^{-2}	2.63×10^{-2}	1.37×10^{-2}	275706
	500	1.34×10^{-2}	2.54×10^{-2}	1.44×10^{-2}	325306
Simple SigFormer	100	3.64×10^{-2}	5.95×10^{-2}	6.08×10^{-2}	6666
	200	3.51×10^{-2}	6.08×10^{-2}	5.88×10^{-2}	6666
	300	3.70×10^{-2}	5.99×10^{-2}	5.87×10^{-2}	6666
	400	3.62×10^{-2}	6.00×10^{-2}	5.99×10^{-2}	6666
	500	4.05×10^{-2}	5.90×10^{-2}	5.96×10^{-2}	6666

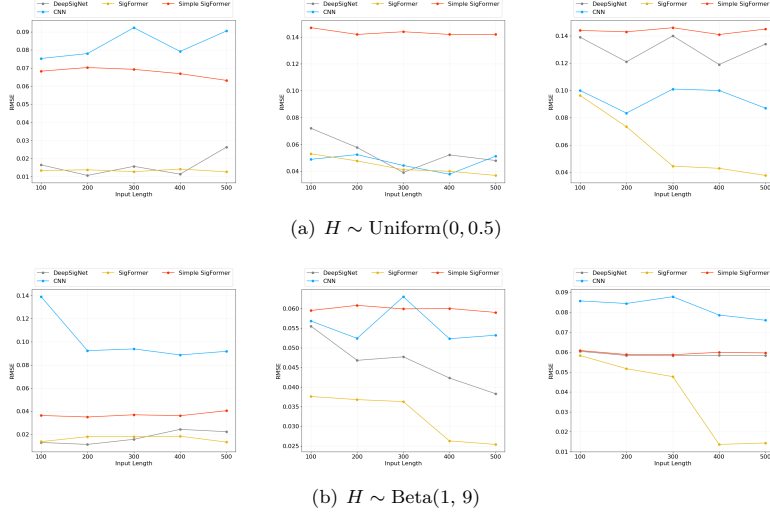


Figure 2: RMSE of calibrating H for different volatility models (Left: fBms; Middle: rBergomi volatility processes; Right: rHeston volatility processes) over input lengths vary from 100 to 500.

4.1.3 Calibrating the Hurst parameter from volatility paths with different signature truncation orders

From the previous example, we have verified the advantages of our SigFormer model in single parameter calibration problems. Following the analysis at the beginning of Section 4.1, we aim to explore the effects of varying signature truncation orders on our SigFormer model next. In this example, we will only use the paths with $H \sim \text{Beta}(1, 9)$ and 100 time steps for simplicity. And we modify the signature truncation order to vary from $\{1, 2, 3, 4, 5\}$ while keeping other settings the same as Section 4.1.1.

The results are shown in Table 4 with the corresponding loss plots shown in Appendix B.2. In order to present the results more clearly, we also plot Table 4 into Figure 4.1.3. It is evident that the optimal signature truncation orders for our SigFormer model vary across different volatility models; specifically, the order is 2 for fBm, 4 for rBergomi, and 5 for rHeston. We also notice that the signature truncation order is correlated with the model complexity, thus we select the truncation order equal to 3 in our SigFormer model for the balance between performance and complexity.

Table 4: Test results for different models with the signature truncation order varying from 1 to 5. Here $H \sim \text{Beta}(1, 9)$ and the input lengths equal to 100.

	Test RMSE			#Params	Truncation order
	fBm	rBergomi	rHeston		
DeepSigNet	4.37×10^{-2}	5.95×10^{-2}	6.08×10^{-2}	4461	1
SigFormer	2.60×10^{-2}	5.95×10^{-2}	6.07×10^{-2}	6006	
Simple SigFormer	5.91×10^{-2}	5.95×10^{-2}	6.08×10^{-2}	1086	
DeepSigNet	8.64×10^{-3}	5.60×10^{-2}	6.05×10^{-2}	5261	2
SigFormer	8.60×10^{-3}	5.05×10^{-2}	6.01×10^{-2}	16781	
Simple SigFormer	5.67×10^{-2}	5.95×10^{-2}	6.08×10^{-2}	2946	
DeepSigNet	1.59×10^{-2}	5.02×10^{-2}	6.03×10^{-2}	9261	3
SigFormer	1.37×10^{-2}	4.26×10^{-2}	5.75×10^{-2}	126906	
Simple SigFormer	4.00×10^{-2}	5.95×10^{-2}	6.08×10^{-2}	6666	
DeepSigNet	3.23×10^{-2}	5.25×10^{-2}	6.01×10^{-2}	29261	4
SigFormer	2.11×10^{-2}	3.61×10^{-2}	5.34×10^{-2}	2083781	
Simple SigFormer	3.09×10^{-2}	5.95×10^{-2}	6.08×10^{-2}	14106	
DeepSigNet	5.74×10^{-2}	4.85×10^{-2}	6.02×10^{-2}	129261	5
SigFormer	8.25×10^{-2}	3.93×10^{-2}	5.24×10^{-2}	47024406	
Simple SigFormer	2.53×10^{-2}	5.95×10^{-2}	6.08×10^{-2}	28986	

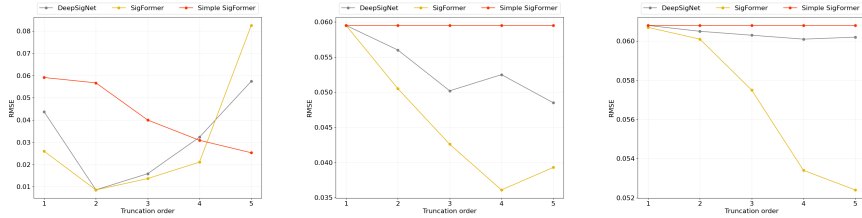


Figure 3: RMSE of calibrating H for different volatility models (Left: fBms; Middle: rBergomi volatility processes; Right: rHeston volatility processes) over signature truncation order varying from 1 to 5.

4.2 Calibrating multiple parameters with rough volatility processes

In previous examples, we have observed that the SigFormer model tends to outperform competing models in more complex calibration problems, such as those involving paths simulated by advanced volatility models and more time steps. Therefore, it is natural to consider applying the SigFormer model in multiple

parameters calibration problems. Furthermore, we also want to simplify the SigFormer model without losing too much accuracy since the complexity of the SigFormer model will increase with the input length as shown in Section 4.1.2.

In this section, we will perform two examples: the first example will verify the advantages of our SigFormer model for multiple parameters calibration problems and the second example will show how to reduce the complexity of the SigFormer model with minimal loss of accuracy.

4.2.1 Implementation details

In order to generate the data sets for multiple parameters calibration problems, we first sample different $\eta, \kappa_1, \kappa_2, \theta$ from the distribution shown in Table 5. Then we simulate rBergomi and rHeston paths with these parameters, as well as $H \sim \text{Beta}(1, 9)$, from Section 4.1.1 to generate four corresponding training/test data sets with the same size as shown in Table 1 and all paths are simulated 100 steps. The first data set consists of rBergomi paths with different η and H . The second data set consists of rHeston paths with different κ_2 and H while keeping $\kappa_1 = 0.1$ and $\theta = 0.3$. The third data set consists of rHeston paths with different κ_1, κ_2 and H while keeping $\theta = 0.3$. The fourth data set consists of rHeston paths with different $\kappa_1, \kappa_2, \theta$ and H . All other settings are kept the same as Section 4.1.1.

Table 5: Distributions for sampling parameters.

Parameters	Distributions	Values
η	Uniform(0, 3)	{0.41, 1.04, 1.56, 2.40, 2.73}
κ_1	Uniform(0, 5)	{0.25, 1.24, 2.83, 3.02, 4.81}
κ_2	Uniform(0, 3)	{0.13, 0.79, 1.58, 2.23, 2.94}
θ	Uniform(0, 0.5)	{0.05, 0.14, 0.21, 0.30, 0.42}

4.2.2 Multiple parameters calibration

We begin this section by calibrating two parameters simultaneously: H, η of rBergomi volatility processes and H, κ_2 of rHeston volatility processes. The results are shown in Table 6 with the corresponding loss plots shown in Appendix B.3.

Table 6: Test results for two parameters calibration with $H \sim \text{Beta}(1, 9)$, $\eta \sim \text{Uniform}(0, 3)$ and $\kappa_2 \sim \text{Uniform}(0, 3)$.

Models	RMSE		#Params
	rBergomi	rHeston	
CNN	0.729	0.786	255202
DeepSigNet	0.728	0.785	9294
SigFormer	0.315	0.422	126939
Simple SigFormer	0.592	0.708	6822

Furthermore, we also perform three and four parameters calibrations for rHeston volatility processes. The results are shown in Table 7 with the corresponding loss plots shown in Appendix B.3. In these multiple calibration problems, we observe that the Simple SigFormer model achieves the same accuracy as the baseline models while requiring fewer model parameters. As for the SigFormer model, it outperforms all other models while requiring more model parameters. These results further demonstrate the advantages of combining self-attention mechanism with the signature transform for calibrating complex rough paths.

Table 7: Test results for three and four parameters calibration with $H \sim \text{Beta}(1, 9)$, $\kappa_1 \sim \text{Uniform}(0, 5)$ and $\kappa_2 \sim \text{Uniform}(0, 3)$.

Models		RMSE of rHeston	#Params
CNN	$\theta = 0.3$	1.400	255331
	$\theta \sim \text{U}(0, 0.5)$	1.180	255460
DeepSigNet	$\theta = 0.3$	1.401	9327
	$\theta \sim \text{U}(0, 0.5)$	1.190	9360
SigFormer	$\theta = 0.3$	0.700	126972
	$\theta \sim \text{U}(0, 0.5)$	0.750	127005
Simple SigFormer	$\theta = 0.3$	1.063	6978
	$\theta \sim \text{U}(0, 0.5)$	0.926	7134

4.2.3 Lower the complexity for the SigFormer model

From the previous example, we have observed that the Simple SigFormer model can achieve great accuracy with fewer model parameters compared to baseline

models, while the SigFormer model outperformed all other models, but it requires a more complex structure. One of the differences between the SigFormer model and the Simple SigFormer model is that the former performs the lifting operation before taking the signature, while the latter does not. As mentioned in Section 4.1.2, the lifting operation adds complexity to the SigFormer model; therefore, modifying the stride of the lifting operation could potentially reduce this complexity with minimal loss of accuracy. In this example, we will only use the paths with $H \sim \text{Beta}(1, 9)$, $\kappa_1 \sim \text{Uniform}(0, 5)$, $\kappa_2 \sim \text{Uniform}(0, 3)$, $\theta \sim \text{Uniform}(0, 0.5)$ and 100 time steps. And we modify the stride of the lifting operation to vary from $\{1, 5, 10, 50\}$ while keeping other settings the same as Section 4.2.2.

The results are shown in Table 8 with the corresponding loss plots shown in Appendix B.4. We observe that an increase in the stride can significantly reduce the number of required model parameters without substantially compromising accuracy.

Table 8: Test results for the SigFormer model with the stride varying from 1 to 50. Here $H \sim \text{Beta}(1, 9)$, $\kappa_1 \sim \text{Uniform}(0, 5)$, $\kappa_2 \sim \text{Uniform}(0, 3)$, $\theta \sim \text{Uniform}(0, 0.5)$ and the input lengths equal to 100.

Models	Stride	RMSE of rHeston	#Params
SigFormer	1	0.745	573405
	5	0.754	176605
	10	0.761	127005
	50	0.774	87325

5 Conclusion

There is a solid theoretical ground for using the signature transform as a tool to calibrate the streams of data. Meanwhile transformers have enjoyed great empirical success in handling sequential data. It is thus natural to combine them together to calibrate volatility processes; in this paper we have constructed a SigFormer model to present how to do this, and also have provided examples to explore the performance of the SigFormer model in different calibration problems.

There are two main contributions in this paper. First, we give the way to combine the signature transform with the self-attention mechanism. Second, we present how to specify the details of the SigFormer model as well as how to simplify it without substantially compromising accuracy.

References

- Hambly, B., and Lyons, T. (2010). Uniqueness for the signature of a path of bounded variation and the reduced path group. *Annals of Mathematics*, 109-167.
- Horvath, B., Muguruza, A. and Tomas, M. (2021). Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models. *Quantitative Finance*, 21, 11–27.
- Kidger, P., Bonnier, P., Perez Arribas, I., Salvi, C., and Lyons, T. (2019). Deep signature transforms. *Advances in Neural Information Processing Systems*, 32.
- Lyons, T. J. (1998). Differential equations driven by rough signals. *Revista Matemática Iberoamericana*, 14(2), 215-310.
- Ma, J. and Wu, H. (2022). A fast algorithm for simulation of rough volatility models. *Quantitative Finance*, 22, 447–462.
- Ma, J., Wu, X., and Wu, H. A neural network rough volatility model. <https://rgdoi.net/10.13140/RG.2.2.30650.98249>
- Mandelbrot, B. B., and Van Ness, J. W. (1968). Fractional Brownian motions, fractional noises and applications. *SIAM review*, 10(4), 422-437.
- Rosenbaum, M. and Zhang, J.F. (2022). Deep calibration of the quadratic rough Heston model. <https://doi.org/10.48550/arXiv.2102.01962>
- Stone, H. (2020). Calibrating rough volatility models: a convolutional neural network approach. *Quantitative Finance*, 20(3), 379-392.
- Tong, A., Nguyen-Tang, T., Lee, D., Tran, T. M., and Choi, J. (2023, November). SigFormer: Signature Transformers for Deep Hedging. In *Proceedings of the Fourth ACM International Conference on AI in Finance* (pp. 124-132).
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser L. and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Wang, S.D. and Hong, L.J. (2021). Option pricing by neural stochastic differential equations: a simulation-optimization approach. 2021 Winter Simulation Conference (WSC), 1–11. doi: 10.1109/WSC52266.2021.9715493

A Architecture of different models

A.1 Architecture of CNN and DeepSigNet model

Here we clarify the specific architecture of CNN and DeepSigNet model as below

- The architecture of CNN model:
 - a convolutional layer with 32 channels and kernel size 20;
 - the leaky ReLu transform with negative slope 0.1;
 - a max pooling layer with size 3;
 - a dropout layer with rate 0.25;
 - a convolutional layer with 64 channels and kernel size 20;
 - the leaky ReLu transform with negative slope 0.3;
 - a max pooling layer with size 3;
 - a dropout layer with rate 0.25;
 - a convolutional layer with 128 channels and kernel size 20;
 - the leaky ReLu transform with negative slope 0.1;
 - a max pooling layer with size 3;
 - a dropout layer with rate 0.4;
 - a dense layer with 128 units;
 - the leaky ReLu transform with negative slope 0.1;
 - a dropout layer with rate 0.3;
 - a dense layer with 1 units;
 - a non-linear transform with sigmoid function.
- The architecture of DeepSigNet model:
 - a convolutional layer with 3 channels and kernel size 3;
 - augmented with time and original values;
 - taking the signature transform with truncation order $N = 3$;
 - a fully connected feedforward neural network with 5 hidden layers, each of size 32 and ReLU activation;
 - a non-linear transform with sigmoid function.
- The architecture of simple SigFormer model:
 - augmented with time;
 - taking the signature transform with truncation order $N = 3$;
 - a single-head self-attention layer;
 - a non-linear transform with sigmoid function.

B Loss plots

B.1 Loss plots for different models with input lengths varying from 100 to 500

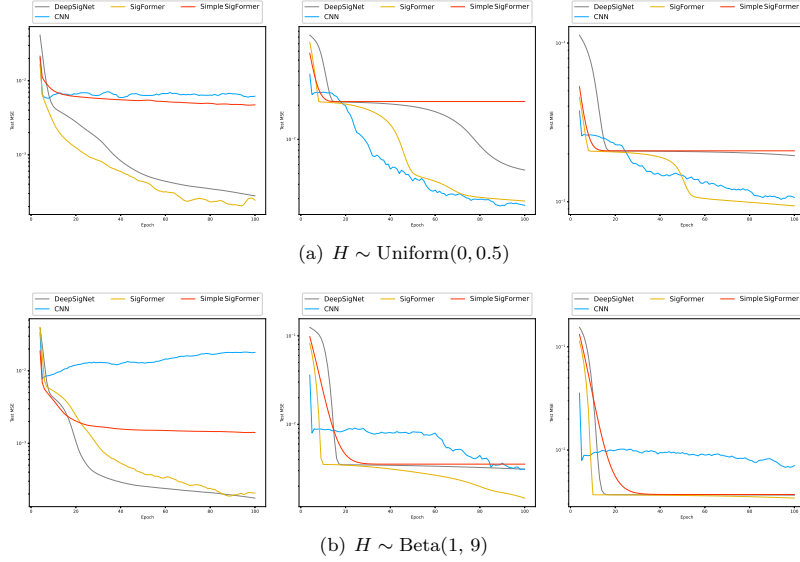


Figure 4: Loss plot for (Left:) fBms; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with input lengths equal to 100. The data has been taken the rolling mean with a window size of 5.

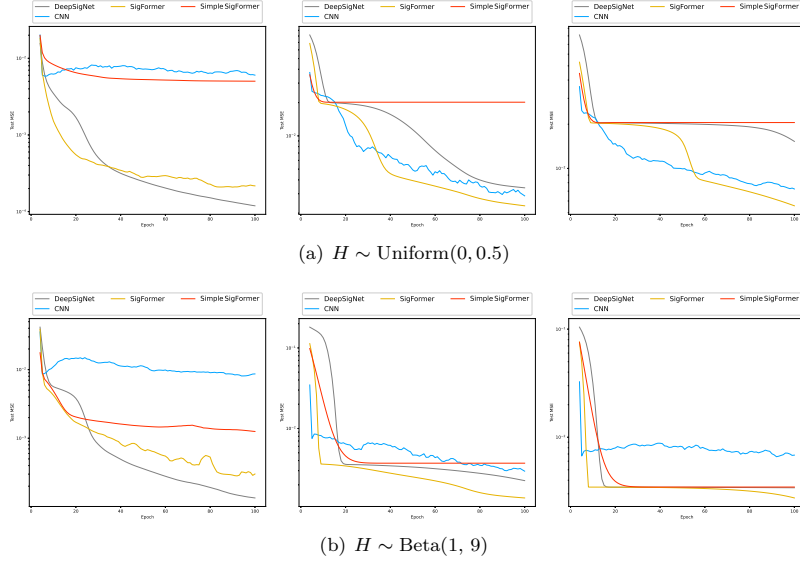


Figure 5: Loss plot for (Left:) fBMs; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with input lengths equal to 200. The data has been taken the rolling mean with a window size of 5.

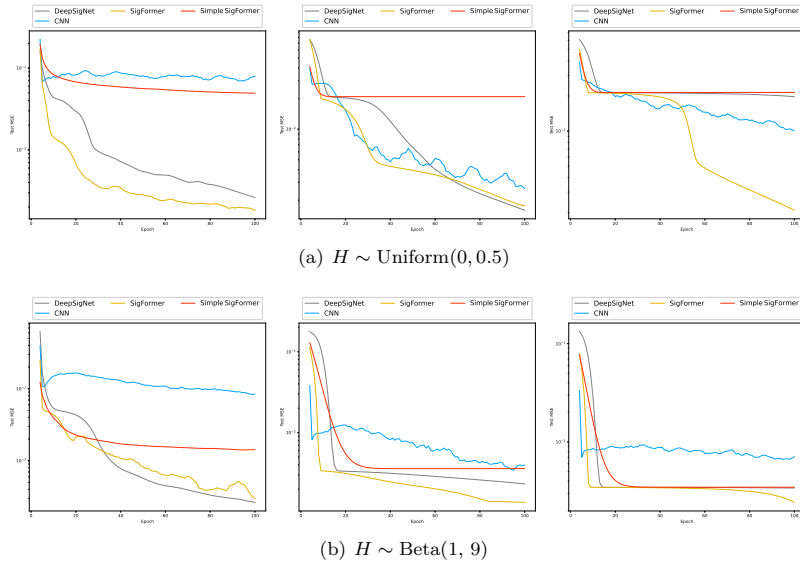


Figure 6: Loss plot for (Left:) fBMs; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with input lengths equal to 300. The data has been taken the rolling mean with a window size of 5.

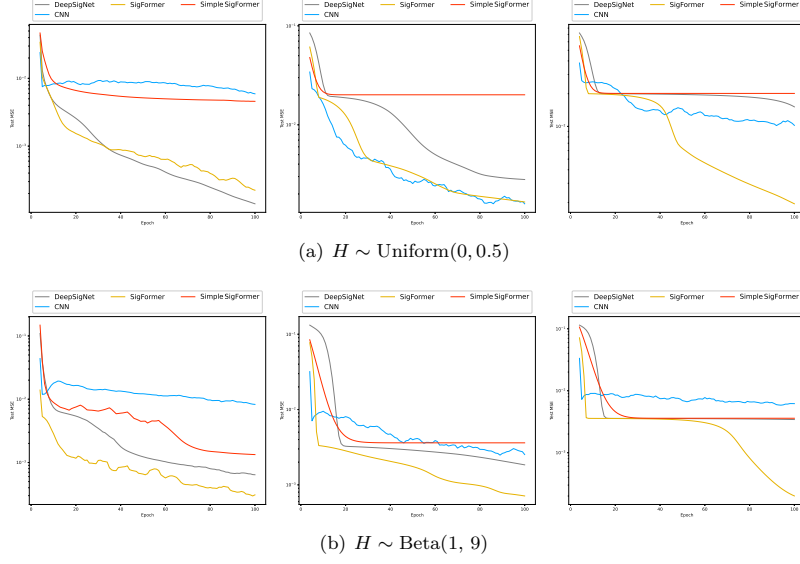


Figure 7: Loss plot for (Left:) fBms; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with input lengths equal to 400. The data has been taken the rolling mean with a window size of 5.

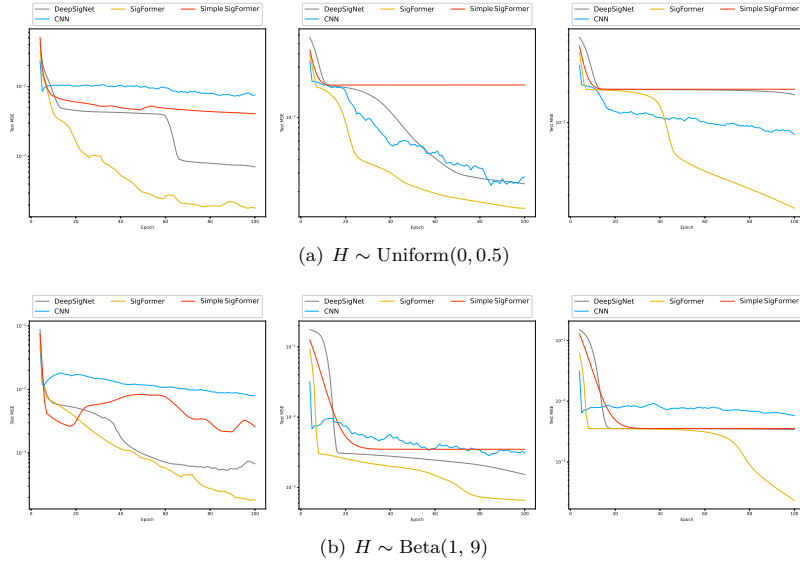


Figure 8: Loss plot for (Left:) fBms; (Middle:) rBergomi volatility processes; (Right:) rHeston volatility processes with input lengths equal to 500. The data has been taken the rolling mean with a window size of 5.

B.2 Loss plots for different models with the signature truncation order varying from 1 to 5

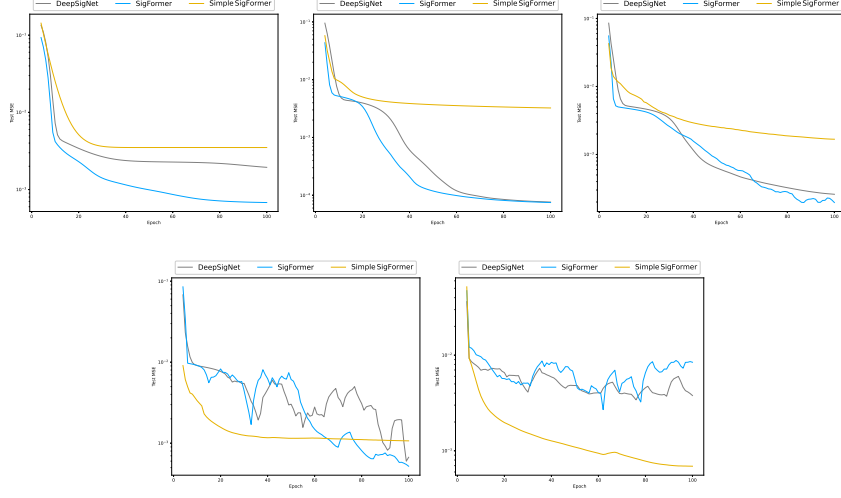


Figure 9: Loss plot for fBms with signature truncation order ranging from (left to right and top to bottom:) $\{1, 2, 3, 4, 5\}$. Here $H \sim \text{Beta}(1, 9)$ and the input lengths equal to 100. The data has been taken the rolling mean with a window size of 5.

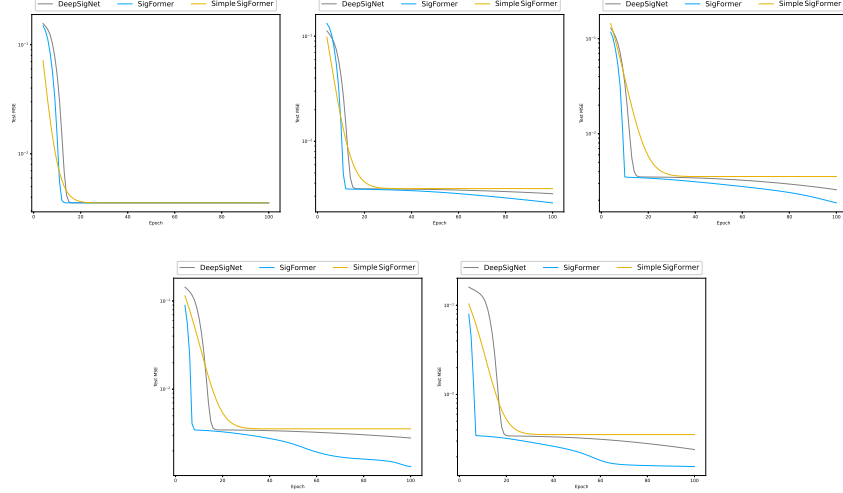


Figure 10: Loss plot for rBergomi volatility processes with signature truncation order ranging from (left to right and top to bottom:) $\{1, 2, 3, 4, 5\}$. Here $H \sim \text{Beta}(1, 9)$ and the input lengths equal to 100. The data has been taken the rolling mean with a window size of 5.

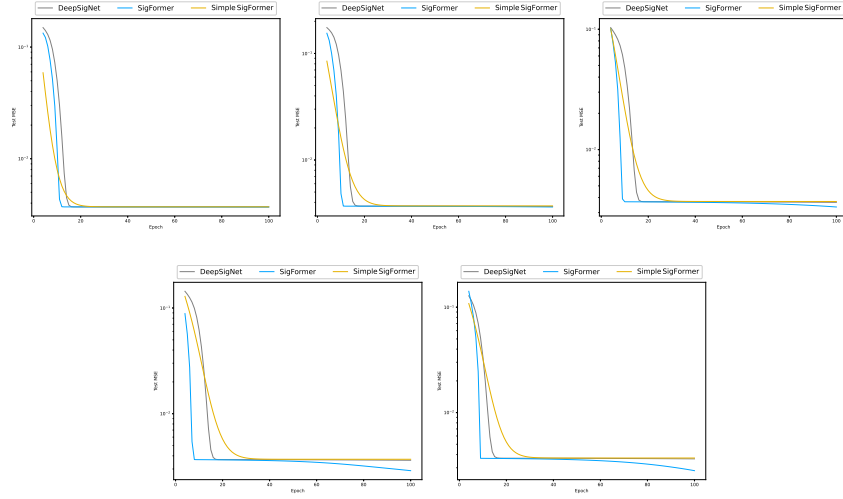


Figure 11: Loss plot for rHeston volatility processes with signature truncation order ranging from (left to right and top to bottom:) $\{1, 2, 3, 4, 5, 6\}$. Here $H \sim \text{Beta}(1, 9)$ and the input lengths equal to 100. The data has been taken the rolling mean with a window size of 5.

B.3 Loss plots for multiple parameters calibrations

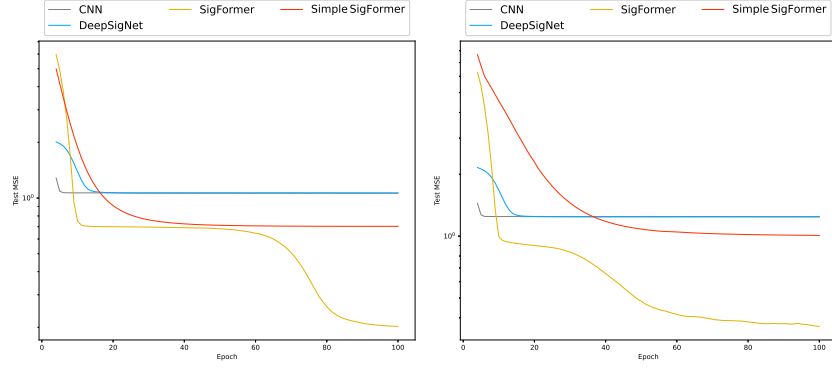


Figure 12: Loss plot of two parameters calibration for (Left:) rBergomi volatility processes and (Right:) rHeston volatility processes with $H \sim \text{Beta}(1, 9)$, $\eta \sim \text{Uniform}(0, 3)$ and $\kappa_2 \sim \text{Uniform}(0, 3)$. The data has been taken the rolling mean with a window size of 5.

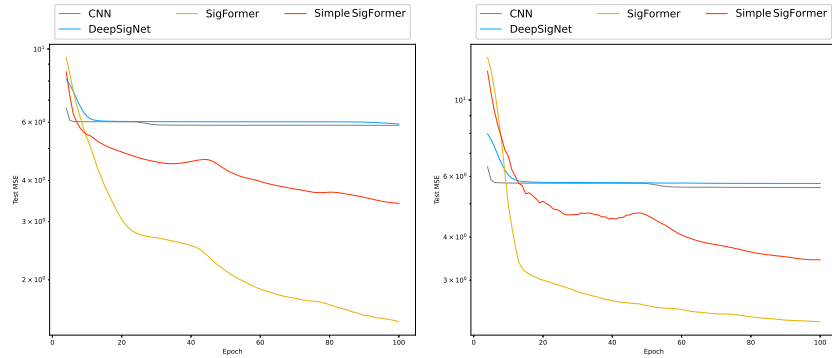


Figure 13: Loss plot of (Left:) three and (Right:) four parameters calibration for rHeston volatility processes with $H \sim \text{Beta}(1, 9)$, $\kappa_1 \sim \text{Uniform}(0, 5)$, $\kappa_2 \sim \text{Uniform}(0, 3)$ and $\theta \sim \text{Uniform}(0, 0.5)$. The data has been taken the rolling mean with a window size of 5.

B.4 Loss plots for the SigFormer model with different complexities

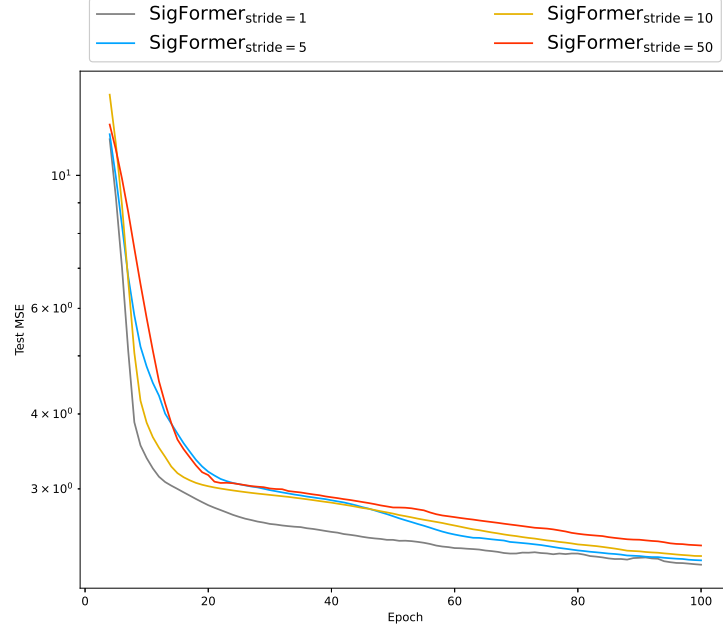


Figure 14: Loss plot for the SigFormer model with the stride varying from 1 to 50. Here $H \sim \text{Beta}(1, 9)$, $\kappa_1 \sim \text{Uniform}(0, 5)$, $\kappa_2 \sim \text{Uniform}(0, 3)$, $\theta \sim \text{Uniform}(0, 0.5)$ and the input lengths equal to 100. The data has been taken the rolling mean with a window size of 5.